

ngspice Linear Solvers — KLU vs. Sparse 1.3

Behavior, defaults, and limitations across DC / AC / transient / noise

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, limitations)	1
TL;DR	1
Selecting and confirming the solver	1
Why KLU rejects noise and pole-zero	2
Two genuine KLU discrepancies	2
The dual-solver test harness	2

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, limitations)

This build of ngspice-46 ships two direct linear solvers, and they are not fully interchangeable across analysis types. This note records exactly how they differ — which is the default, how to select and confirm each, and where KLU falls short — based on a direct read of the source and a solver-by-solver sweep of the whole [examples/](#) suite.

TL;DR

	KLU	Sparse 1.3
Availability in this build	Compiled in (SuiteSparse, statically linked — 44 <code>klu_*</code> symbols in the binary)	Always present (ngspice's own solver)
Default?	No	Yes — this build defaults to Sparse 1.3
How to select	<code>.option klu</code>	<code>.option sparse</code> (or just the default)
DC op / DC sweep	✓ correct (one caveat below)	✓ correct
AC	✓ correct	✓ correct
Transient	✓ correct (one caveat below)	✓ correct
Noise (.noise)	✗ rejected — <code>noisean.c:78</code>	✓ correct
Pole-zero (.pz)	✗ rejected — <code>pzan.c:31</code>	✓ correct

Practical guidance: leave the default (Sparse 1.3) unless you have a specific reason to switch. Sparse 1.3 runs every analysis in the suite — DC, AC, transient, *and* noise/pole-zero. Reach for `.option klu` on large, sparse DC/AC problems where KLU's ordering and factorization are faster, but do not use it for noise or pole-zero (ngspice will error out), and expect it to be less robust on stiff transient edges and degenerate topologies (see the two genuine discrepancies below).

Selecting and confirming the solver

The netlist option is a simple flag:

```
.option klu      * use KLU (SuiteSparse)
.option sparse   * use the legacy Sparse 1.3 (this is also the default)
```

Because unknown `.option` keywords are silently ignored by ngspice, "it ran without complaint" is *not* evidence the solver changed. To confirm which solver is actually active, use the interactive option command, which prints the live CKTkluMODE:

```
.control
run
option      * prints, among other things: "Matrix solver: KLU" or "Sparse 1.3"
.endc
```

On the committed bin/ binary this reports **Sparse 1.3** for a bare deck and for `.option sparse`, and **KLU** only when `.option klu` is given — confirming Sparse 1.3 is the default here.

Why KLU rejects noise and pole-zero

These are not silent wrong answers — ngspice refuses the analysis with an explicit diagnostic and aborts:

```
Error: Noise simulation is not (yet) supported with 'option KLU'.      (noisean.c:78)
Error: Pole/zero analysis is not (yet) supported with 'option KLU'.    (pzan.c:31)
```

The noise and pole-zero paths build and manipulate the small-signal matrix in a way that was only wired up for the Sparse 1.3 storage, not the KLU/CSC path. This is an upstream ngspice limitation, independent of the Verilog-A/OSDI work in this repository. The `.noise` and `.pz` engines themselves are correct — they simply require the Sparse 1.3 solver.

Two genuine KLU discrepancies

Beyond the noise/pole-zero refusals, two examples in the suite produce a different (wrong) result under KLU than under Sparse. Both are marked KLU_XFAIL in the [dual-solver harness](#) so they are exercised and tracked rather than silently skipped.

1. `opamp741` — stiff transient diverges. The transistor-level [opamp741 example](#) (a μ A741 from ~70 lines of Verilog-A BJT):

- Sparse 1.3: the large-signal slew test (`pulse(-5 5 ...)` into the follower) runs the full `tran 20n 80u` cleanly (~4058 points), slew rate ≈ 0.54 V/ μ s.
- KLU: the same run diverges at the slewing edge — `v(out)` overshoots past the -5 V rail to ≈ -6.16 V and ngspice aborts the timestep at $t \approx 2.03$ μ s (~133 points), so the characterization can't complete.

A convergence/timestep robustness difference on a stiff circuit: Sparse's pivoting survives the fast edge and KLU's does not. (E-111's line search does not help — it damps the DC-op Newton, not per-timepoint transient iterations.)

2. `groundcontrib` — wrong DC answer on a degenerate topology. The [groundcontrib example](#) drives a single node `p` with a node-to-ground voltage contribution `V(p) <+ 1.5` through a load resistor to ground. On this exact same deck, changing only the solver:

```
.option sparse -> v(p) = 1.5    (correct)
.option klu     -> v(p) = 0.0    (wrong)
```

This is a genuine KLU correctness miss (not a refusal or a divergence): on this degenerate single-node system KLU returns zero. It is the reason to prefer Sparse for unusual/degenerate one-node topologies. The ~90 other OSDI examples run correctly under KLU, so the issue is specific to this topology, not OSDI generally.

The dual-solver test harness

Every `verify_*.py` in [examples/](#) is run under both solvers automatically. Each script calls `check_both_solvers(__file__)` (right after importing `_setup`); on a normal run that re-executes the script once per solver — injecting `.option sparse` / `.option klu` into every ngspice deck — and prints a combined verdict:

```
=== BOTH-SOLVER RESULT [ceil]: sparse=PASS klu=PASS => OK ===
```

KLU's unsupported analyses (noise / pole-zero) are auto-detected and reported SKIP; the two KLU_XFAIL examples above are expected-fail under KLU. Env escape hatches: `NGSPICE_SOLVER=klu` | `sparse` runs once under one solver; `NG_BOTH=0` disables the dual run.

Sweep result (all 101 machine-checkable scripts): every example is `sparse=PASS` with `klu` in `{PASS, SKIP, XFAIL}` — i.e. the two solvers agree wherever KLU is applicable — with two standing exceptions unrelated to the solver:

hierbranch fails under both solvers (the prebuilt bin/ ngspice predates the E-86 hierarchical-branch-probe feature), so it is a binary-version gap, not a solver difference.

Conclusion: for DC, AC, and transient the two solvers agree across the suite, with exactly two KLU exceptions (the stiff opamp741 transient, and the degenerate groundcontrib DC topology), while noise and pole-zero are Sparse-1.3-only by ngspice design. Sparse 1.3, the default, covers everything — which is why it is the default and why the dual-solver harness keys correctness off it.